

多参 / 睡眠 / 肌电算法结果展示与报告预览前端

企业 Demo 前端，用于展示多参数生命体征、睡眠分期/事件、肌电算法结果，并提供图表分析、数据导入和报告预览能力。当前使用 Mock 数据，后续可直接切换到 FastAPI/Flask 后端接口。

技术栈

- React + Vite
- TypeScript
- React Router
- Ant Design
- ECharts
- Axios
- ESLint
- Prettier

运行方法

进入项目根目录：

```
cd /home/kody304/实习
```

首次运行安装依赖：

```
npm install
```

启动本地开发服务：

```
npm run dev
```

默认开发服务地址：

```
http://localhost:5173
```

如果依赖已经安装过，后续只需要执行：

```
npm run dev
```

生产构建：

```
npm run build
```

代码检查：

```
npm run lint
npm run typecheck
```

格式化：

```
npm run format
```

目录说明

src/	
api/	Axios 实例与后端接口封装
api/adapters/	后端 DTO 到前端领域模型的兼容适配
assets/	静态资源预留目录
charts/	ECharts 基础组件与 option 构造函数
components/	通用 UI 组件
features/	业务域扩展层，例如算法域、图表注册表
hooks/	业务 Hook
layouts/	Header + Sidebar + Main 统一布局
mock/	Mock 数据
pages/	页面组件
report/	报告预览专用组件
router/	React Router 配置
services/	页面到 API 的服务层
styles/	全局样式
types/	全局 TypeScript 类型
utils/	格式化、统计、文件解析等工具

已实现页面

- 首页：项目状态、患者与检测概览
- 数据导入：支持 JSON / CSV 文件解析、错误提示、数据预览
- 项目选择：预留多项目管理入口
- 算法展示：多参、睡眠、肌电三类 Tab，含 Card、Table、自动统计
- 图表分析：多参趋势、睡眠时间轴、睡眠比例、事件柱状图、EMG 波形、RMS、FFT
- 报告预览：患者信息、检测信息、统计指标、图表、结论、医生建议、浏览器打印导出 PDF

如何新增页面

1. 在 `src/pages/` 新建页面组件。

2. 在 `src/router/index.tsx` 使用 `lazy` 添加路由。
3. 如需进入侧边栏，在 `src/router/navigation.tsx` 添加导航项。
4. 复杂页面优先拆出 `components/` 或独立业务目录。

示例：

```
// src/pages/NewPage.tsx
import PageTitle from '@components/PageTitle';

export default function NewPage() {
  return <PageTitle title="新页面" description="页面说明" />;
}
```

如何新增算法展示

新增 ECG、EEG、PPG 等算法时，推荐按以下顺序扩展：

1. 在 `src/types/index.ts` 定义算法记录类型，并加入 `AlgorithmDataset`。
2. 在 `src/mock/algorithm.ts` 添加不少于 20 条 Mock 记录。
3. 在 `src/utils/statistics.ts` 添加统计函数。
4. 在 `src/features/algorithm/domainRegistry.tsx` 添加一个 `AlgorithmDomainConfig`。

页面 `src/pages/AlgorithmPage.tsx` 不需要新增重复 Tab 代码。

如何新增图表

1. 在 `src/charts/options.ts` 新增 `buildXxxOption`。
2. 在 `src/features/charts/chartRegistry.ts` 注册图表标题、高度、栅格和 option 构造函数。
3. 数据转换放在 option 构造函数或 `utils/` 中。

```
<BaseChart option={buildMultiParameterLineOption(records)} height={360} />
```

如何新增接口

1. 在 `src/types/index.ts` 定义请求/响应类型。
2. 在 `src/api/` 新建接口文件，使用 `ApiClient`。
3. 如果后端字段未稳定，在 `src/api/adapters/` 写 `normalizer`。
4. 在 `src/services/` 暴露页面使用的服务函数。
5. Mock 模式下使用 `useMockApi` 分支，真实模式下调用后端。

接口层已处理 FastAPI 常见的 `detail` 校验错误，并会给请求附加 `X-Request-Id`，方便后端日志排查。

后端结构暂不确定时

前端展示层固定使用 `AlgorithmDataset`、`PatientInfo` 等领域类型。后端可以先返回 `snake_case`、`camelCase`、`data/result` 包装或不同数组路径，接口 adapter 会尽量归一化。

后端最终结构变化时，优先改：

```
src/api/adapters/algorithmAdapter.ts
```

不要把字段兼容逻辑散落到页面、表格和图表里。

部署方法

```
npm run build
```

将 **dist/** 部署到 Nginx、静态对象存储或企业前端平台。若后端 API 地址不是同域，设置环境变量：

```
VITE_API_BASE_URL=https://api.example.com  
VITE_USE MOCK=false
```

Nginx 单页应用需要把未知路径回退到 **index.html**。

生产构建已启用路由懒加载、ECharts 按需注册和 vendor 分包。当前 **npm run build** 输出无 warning 和 error。